Copy Guide - Assignment No. 1, Parallel and Distributed Computing

Name: Umair Hakeem Roll No.: 3914 NUML Islamabad Instructor: Dr. Sehrish Amjad

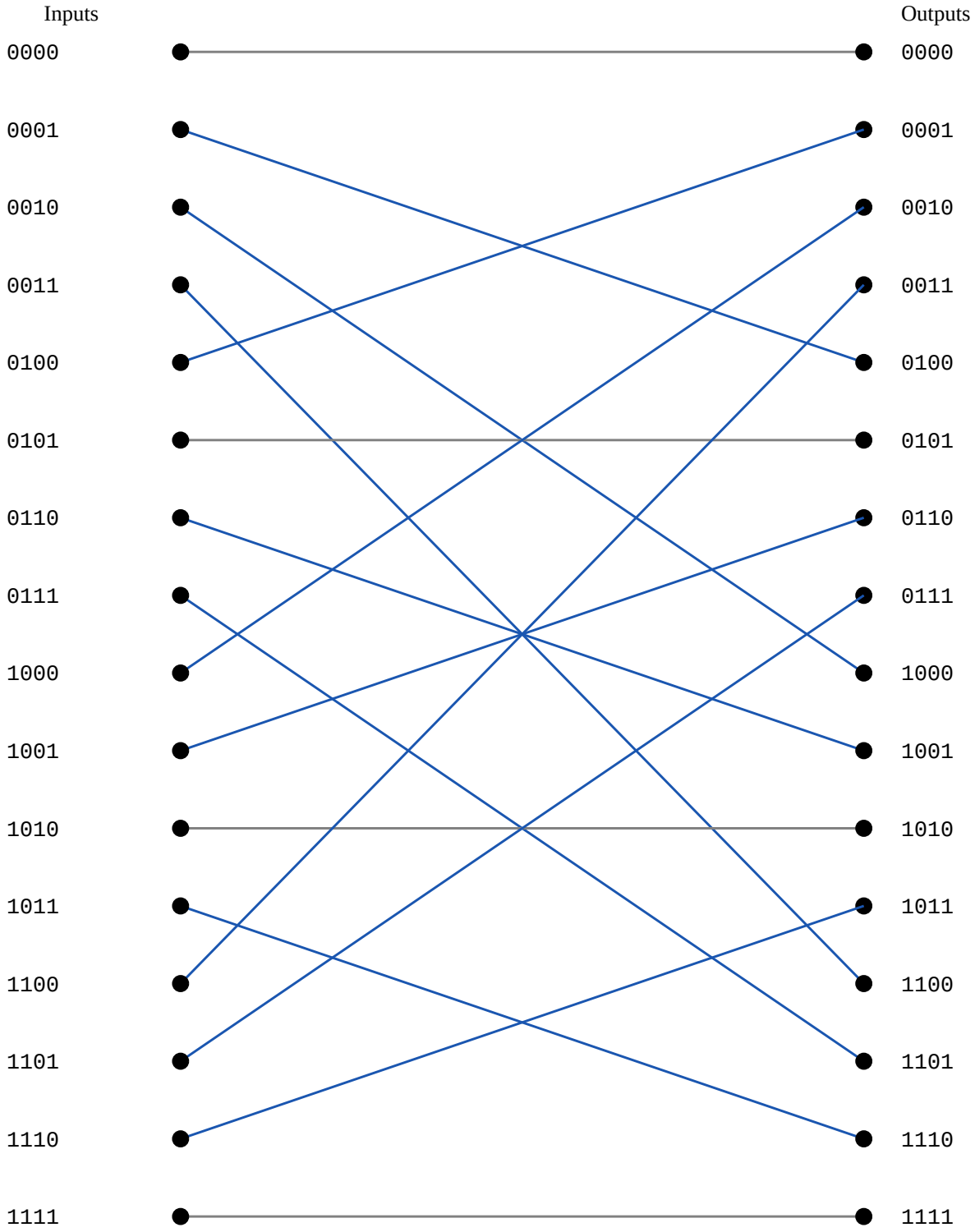
This is a study and copy guide: write each answer out in your own handwriting. Write "Q No. X" before each answer. Page-by-page: Page 1-2 = Q1, Page 3 = Q2 and Q3, Page 4 = Q4, Page 5 = Q5.

Q No. 1: Perfect shuffle on a 16 x 16 Omega network (left rotate by 2 bits) (5 marks)

Given: 16 inputs, so n = 4 address bits. Stages = log2(16) = 4, switches per stage = 16/2 = 8 (32 in total). Each switch is pass-through (straight) or cross-over. Rule: left_rotate by 2 bits: a3 a2 a1 a0 becomes a1 a0 a3 a2.

Input	left_rotate(input)	Output
0000	left_rotate(0000) = 0000	0
0001	left_rotate(0001) = 0100	4
0010	left_rotate(0010) = 1000	8
0011	left_rotate(0011) = 1100	12
0100	left_rotate(0100) = 0001	1
0101	left_rotate(0101) = 0101	5
0110	left_rotate(0110) = 1001	9
0111	left_rotate(0111) = 1101	13
1000	left_rotate(1000) = 0010	2
1001	left_rotate(1001) = 0110	6
1010	left_rotate(1010) = 1010	10
1011	left_rotate(1011) = 1110	14
1100	left_rotate(1100) = 0011	3
1101	left_rotate(1101) = 0111	7
1110	left_rotate(1110) = 1011	11
1111	left_rotate(1111) = 1111	15

Q No. 1 (continued): Shuffle diagram



Observations: 0000, 0101, 1010, 1111 map to themselves (straight lines, pass-through). The rest swap in pairs: 1 and 4, 2 and 8, 3 and 12, 6 and 9, 7 and 13, 11 and 14, so those lines cross and the switches are set to cross-over. Draw the lines in the order of the table; pass-through switches are the self-mapped inputs, cross-over for the others.

Q No. 2: Overall speedup and runtime (T = 200 s) (5 marks)

Formula (Amdahl, generalised): Speedup = 1 / [sum of (f / s)], f = fraction of original time, s = speedup of that part.

Assumption (the paper gives no speedup values - confirm with your instructor): database access not improved (s = 1), numerical computation 10x faster, report generation 2x faster.

Part	Fraction	Original	s	New time
Database access	0.20	40 s	1	40 s
Numerical computation	0.50	100 s	10	10 s
Report generation	0.30	60 s	2	30 s
Total	1.00	200 s		80 s

Speedup = 1 / (0.20/1 + 0.50/10 + 0.30/2) = 1 / (0.20 + 0.05 + 0.15) = 1 / 0.40 = **2.5**
Expected runtime = 200 / 2.5 = **80 s** (time saved = 120 s).

Q No. 3: Overall speedup and time saved (5 marks)

Component	Original	Improvement	New time
Serial initialization	20 s	none	20 s
Data transfer	20 s	2x	20/2 = 10 s
Parallel kernel	50 s	10x	50/10 = 5 s
Result processing	10 s	4x	10/4 = 2.5 s
Total	100 s		37.5 s

Overall speedup = 100 / 37.5 = **2.67**
Time saved = 100 - 37.5 = **62.5 s** (62.5 % reduction).

Q No. 4: Flynn's taxonomy: execution time and speedup (10 marks)

N = 1,000,000 items, 1 instruction = 1 microsecond, one operation per item, no memory/cache/sync/communication cost. Baseline = SISD.

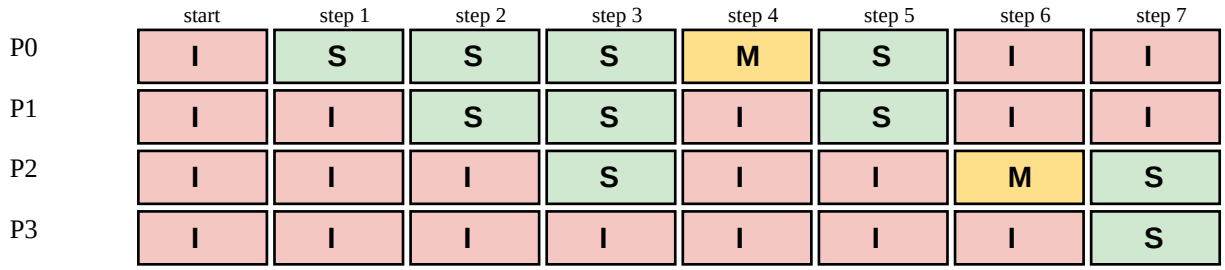
Category	Working	Time	Speedup
SISD (1 instruction stream, 1 data stream)	1,000,000 x 1 us	1,000,000 us = 1 s	1
SIMD (width 8)	1,000,000 / 8 = 125,000 steps x 1 us	125,000 us = 0.125 s	1 / 0.125 = 8
MIMD (4 processors, 4 different tasks)	all four run at the same time, each handles N items: time = max = 1,000,000 x 1 us	1,000,000 us = 1 s	SISD needs 4 x 1 s = 4 s, so 4 / 1 = 4
MISD (3 stages: scaling, squaring, validation)	pipelined: N + (3 - 1) = 1,000,002 steps	1,000,002 us = approx 1 s	SISD needs 3 x 1 s = 3 s, so 3 / 1.000002 = approx 3

Comparison: SIMD gives the best speedup (8x) for this loop because the loop is data-parallel: one instruction is applied to many data items. MIMD (4x) and MISD (approx 3x) are faster only compared with running their tasks one after another; they do different work at the same time and do not suit a simple element-wise loop. SISD is the sequential baseline (speedup 1).
Note: if the 4 MIMD processors only split this loop, time = 1,000,000 / 4 = 250,000 us = 0.25 s (speedup 4).

Q No. 5: MSI cache coherence for block X (10 marks)

Costs: Crd = 20, Cinv = 10, Cdata = 40, Lmem = 80 cycles. Read miss from memory = Crd + Lmem + Cdata = 140. Read miss served by another cache = Crd + Cdata = 60. Upgrade S to M (BusInv) = Cinv = 10. Write miss (BusRdX) from memory = Cinv + Lmem + Cdata = 130. Start: only memory has X; P0, P1, P2, P3 = I, I, I, I.

#	Access	Bus transaction	States after (P0 P1 P2 P3)	Data source / action	Cost
1	P0: read X	BusRd	S I I I	Memory supplies block	20+80+40 = 140
2	P1: read X	BusRd	S S I I	Memory supplies block	20+80+40 = 140
3	P2: read X	BusRd	S S S I	Memory supplies block	20+80+40 = 140
4	P0: write X = 5	BusInv (upgrade)	M I I I	No data needed. P1, P2 invalidated	Cinv = 10
5	P1: read X	BusRd	S S I I	P0 supplies data, writes back to memory (M to S)	20+40 = 60
6	P2: write X = 7	BusRdX	I I M I	Memory supplies block. P0, P1 invalidated	10+80+40 = 130
7	P3: read X	BusRd	I I S S	P2 supplies data, writes back (M to S)	20+40 = 60
	Total				**680 cycles**



M = Modified (yellow), S = Shared (green), I = Invalid (red). Copy as a table with rows P0-P3 and columns start, step 1 to 7.